

WriteUp HackforChange2 CTF [2026]



Nama: Hasel Aristarchus Silitonga
Username: CTF Jalur Doa
Email: hasel.silitonga69@gmail.com

Daftar Isi

[Foren].....	1
[Slum Survey Photo].....	1
[Welfare Payment PCAP].....	1
[Reverse Engineering].....	1
[Basic Income Crackme].....	1

[Foren]

[Slum Survey Photo]

Tingkat Kesulitan: Easy

Goal: Untuk mendapatkan flag dan menyelesaikan challenge dengan mencari flag dalam gambar

Langkah Penyelesaian:

Setelah masuk ke challenge, kita harus download gambar yang terlihat sebagai berikut



Bila kita strings gambar ini, maka terlihat metadata dan flag berikut

Flag: SDG{c19d24eb9f0a668bddad43a61666c784}

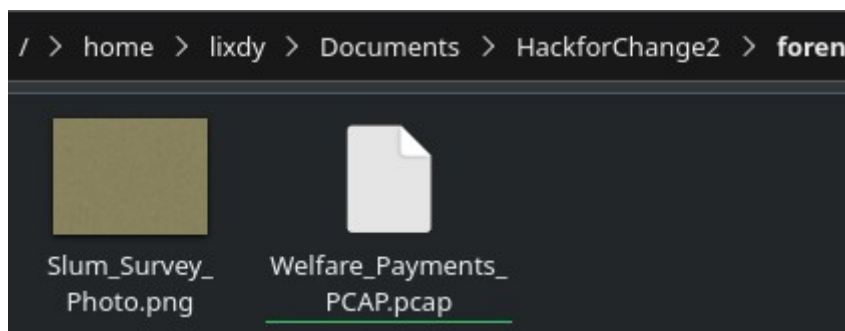
[Welfare Payment PCAP]

Tingkat Kesulitan: Easy

Goal: Untuk mendapatkan flag dan menyelesaikan challenge dari analisis file PCAP

Langkah Penyelesaian:

Setelah mengunduh file .pcap dari web, terlihat ada keterangan pembayaran dari berbagai macam kategori.



10.2.984000	10.0.5.21	10.0.5.10	MQTT	150 Publish Message	[payments/centra/audit]
17.6.891000	10.0.5.21	10.0.5.10	MQTT	151 Publish Message	[payments/centra/audit]
18.3.134000	10.0.5.21	10.0.5.10	MQTT	221 Publish Message	[payments/centra/disbursement]
19.3.383000	10.0.5.21	10.0.5.10	MQTT	223 Publish Message	[payments/northern/disbursement]
20.3.458000	10.0.5.21	10.0.5.10	MQTT	222 Publish Message	[payments/centra/disbursement]
21.4.955000	10.0.5.21	10.0.5.10	MQTT	223 Publish Message	[payments/northern/disbursement]
22.4.380000	10.0.5.21	10.0.5.10	MQTT	42 Disconnect Req	

- Frame 17: 116 bytes on wire (928 bits), 116 bytes captured (928 bits) - Raw packet data - Internet Protocol Version 4, Src: 10.0.5.21, Dst: 10.0.5.10 - Transmission Control Protocol, Src Port: 48759, Dst Port: 1883, Seq: 1804, Ack: 5, Len: 76 - MQ Telemetry Transport Protocol, Publish Message - Header Flags: 0x30, Message Type: Publish Message, QoS Level: At most once delivery (Fire and Forget) - Msg Len: 74 - Topic Length: 20 - Topic: payments/admin/audit - Message: 55395248657a466c5954497a4d574978596d4a6b5a5445314e574a6c59575a6d597a566d4e6d56c4d44646b59545a6c60513d3d	<pre> 0000 45 00 00 74 1c 5a 40 00 40 06 00 0c 0a 00 05 15 E..t.Z0.0..... 0010 0a 00 05 0a be 77 07 5b 68 92 bd fa 0d b0 01 3d w[h.....= 0020 50 18 7f ff 91 43 00 00 30 4a 00 14 70 61 79 6d P....C.0J..paym 0030 65 6e 74 73 2f 61 64 6d 69 6e 2f 61 75 64 69 74 ents/ade in/audit 0040 55 39 52 48 65 7a 46 6c 59 54 49 7a 4d 57 49 78 U0RHezFLYTizMWIx 0050 59 6d 4a 6b 5a 54 45 31 4e 57 4a 6c 59 57 5a 6d YmJkZTE1NWJlYWZm 0060 59 7a 56 6d 4e 6d 56 6c 4d 44 64 6b 59 54 5a 6c YzVmNmVlMDdkYTZl 0070 66 51 3d 3d fQ== </pre>
--	--

Bila dianalisis di bagian admin ada kode base64 seperti berikut, yang bila kita decode berisi flag di bawah.

```

[lixdy@parrot]~$ echo "U0RHezFLYTizMWIxYmJkZTE1NWJlYWZmYzVmNmVlMDdkYTZlfQ==" | base64 -d
SDG{1ea231b1bbde155beaffc5f6ee07da6e}

```

Flag: SDG{1ea231b1bbde155beaffc5f6ee07da6e}

[Reverse Engineering]

[BasicIncome_Crackme]

Tingkat Kesulitan: Medium

Goal: Untuk mendapatkan flag dan menyelesaikan challenge dari analisis kode

Langkah Penyelesaian:

Setelah kita download file BasicIncome_Crackme dari web, terlihat bila kita strings, ini merupakan file ELF yang menganalisis voucher yang valid

```
__PAGEZERO
__TEXT
__text
__TEXT
__stubs
__TEXT
__cstring
__TEXT
__const
__TEXT
__unwind_info
__TEXT
__DATA_CONST
__got
__DATA_CONST
__LINKEDIT
/usr/lib/dyld
/usr/lib/libSystem.B.dylib
Th"@9
Th&@9
Th*@9
Th.@9
Th2@9
Th6@9
Th:@9
Th>@9
ThB@9
> K)=@
usage: %s <password>
nope
flag.png
fopen
developed flag.png
```

Sayangnya, struktur kode kurang jelas bila menampilkan ini saja. Maka kita beralih ke Ghidra untuk menganalisis lebih lanjut hingga ketemu function entry dan kode berikut

The screenshot shows the Ghidra interface with the assembly view on the left and the decompiled code on the right. The assembly view shows instructions for the 'entry' function, including MOV, JMP, and CALL. The decompiled code shows the 'processEntry' function, which is called by 'entry'. The 'processEntry' function has a while loop that calls 'FUN_00402b20' and has a warning about an infinite loop.

```

*****
*                               FUNCTION
*****
undefined processEntry entry()
<UNASSIGNED> <RETURN>
Stack[-0x10]:8 local_10
entry
XREF[5]:

```

Namun bila dilihat di fungsi 00401680, diketahui bahwa ini adalah fungsi main dari kode dan ada string menarik sebagai

```

2 undefined8 FUN_00401680(void)
3
4 {
5     char cVar1;
6     long lVar2;
7     long lVar3;
8     undefined8 uVar4;
9     byte *pbVar5;
10    byte bVar6;
11    byte local_88 [47];
12    char cStack_59;
13    undefined1 local_58 [16];
14    undefined1 local_48 [16];
15    undefined1 local_38 [16];
16    undefined1 local_28 [16];
17    undefined1 local_18 [16];
18
19    bVar6 = 0;
20    local_58 = (undefined1 [16])0x0;
21    local_48 = (undefined1 [16])0x0;
22    local_38 = (undefined1 [16])0x0;
23    local_28 = (undefined1 [16])0x0;
24    local_18 = (undefined1 [16])0x0;
25    FUN_0040acf0("BasicIncome eligibility calculator\n",1,0x23,PTR_DAT_004aa6c8);
26    FUN_0040acf0("household_id> ",1,0xe,PTR_DAT_004aa6c8);
27    FUN_0040a960(PTR_DAT_004aa6c8);
28    lVar2 = FUN_0040aa50(local_58,0x50,PTR_DAT_004aa6d8);
29    if (lVar2 != 0) {
30        lVar2 = thunk_FUN_004193c0(local_58);
31        while (lVar2 != 0) {
32            lVar3 = lVar2 + -1;
33            cVar1 = local_58[lVar3];
34            if ((cVar1 != '\n') && (cVar1 != '\r')) {
35                if (lVar2 == 0x10) {
36                    lVar2 = 0;
37                    do {
38                        if ((&DAT_0047e0d0)[lVar2] != ((&DAT_0047e0e0)[(uint)lVar2 & 7] ^ local_58[lVar2])) {
39                            FUN_0040acf0("Ineligible.\n",1,0xc,PTR_DAT_004aa6c8);
40                            return 1;
41                        }
42                        lVar2 = lVar2 + 1;
43                    } while (lVar2 != 0x10);

```

berikut

```
    pbVar5 = local_88;
    for (lVar2 = 0x29; lVar2 != 0; lVar2 = lVar2 + -1) {
        *pbVar5 = 0;
        pbVar5 = pbVar5 + (ulong)bVar6 * -2 + 1;
    }
    lVar2 = 0;
    do {
        local_88[lVar2] = local_58[(uint)lVar2 & 0xf] ^ (&DAT_0047e0a0)[lVar2];
        lVar2 = lVar2 + 1;
    } while (lVar2 != 0x28);
    FUN_0040acf0("Eligible. Voucher: ",1,0x13,PTR_DAT_004aa6d0);
    uVar4 = thunk_FUN_004193c0(local_88);
    FUN_0040acf0(local_88,1,uVar4,PTR_DAT_004aa6d0);
    FUN_0040cd90(10,PTR_DAT_004aa6d0);
    return 0;
}
break;
}
local_58[lVar3] = 0;
lVar2 = lVar3;
}
FUN_0040acf0("Ineligible: household_id must be 16 characters.\n",1,0x30,PTR_DAT_004aa6c8);
}
return 1;
}
```

Di dalam fungsi berikut, ditemukan logika validasi:

```
if ((&DAT_0047e0d0)[i] != ((&DAT_0047e0e0)[i % 8] ^ local_58[i]))
```

Dari sini bisa dibaca rumusnya:

```
DAT_0047e0d0[i] == DAT_0047e0e0[i % 8] XOR input[i]
```

Dibalik untuk mendapat input yang valid:

```
input[i] = DAT_0047e0d0[i] XOR DAT_0047e0e0[i % 8]
```

Selain itu ditemukan syarat penting: input harus tepat 16 karakter, jika tidak program langsung print 'Ineligible: household_id must be 16 characters.'

Oleh karena itu, bisa kita tentukan melalui serangkaian langkah berikut untuk mencari password voucher yang benar

```
[x]-[lixdy@parrot]-[/opt/ghidra_12.2_DEV] reverse engineering >
$python3 << 'EOF'
d0=bytes.fromhex("de18ae735fbd79188e4bf87258eb7d18")
e0 = bytes.fromhex("de18ae735fbd7918")
overview challenge lengkap dengan tabel info binary
password = ""
for i in range(16):
    password += chr(d0[i] ^ e0[i % 8])
print("Password: ", password)
EOF
Password: PSVV
[lixdy@parrot]-[/opt/ghidra_12.2_DEV]
$python3 << 'EOF'
d0=bytes.fromhex("de18ae735fbd79188e4bf87258eb7d18")
e0 = bytes.fromhex("de18ae735fbd7918")
password=bytes([d0[i]^e0[i%8] for i in range(16)])
print("Raw:", password)
print("Hex:", password.hex())
EOF
Raw: b'\x00\x00\x00\x00\x00\x00\x00\x00\x00PSV\x01\x07V\x04\x00'
Hex: 0000000000000000000005053560107560400
[lixdy@parrot]-[/opt/ghidra_12.2_DEV]
$python3 << 'EOF'
d0 = bytes.fromhex("de18ae735fbd79188e4bf87258eb7d18")
e0 = bytes.fromhex("de18ae735fbd7918")

# Coba: input[i] = d0[i] XOR e0[i % 8] sudah dicoba
# Sekarang coba: cari pattern lain

# Lihat apakah d0 dan e0 punya hubungan
print("d0[:8]:", d0[:8].hex())
print("e0      :", e0[:8].hex())
print("Sama?", d0[:8] == e0)
EOF
d0[:8]: de18ae735fbd7918
e0      : de18ae735fbd7918
Sama? True
```

```
[lixdy@parrot]-[~/Documents/HackforChange2/rev]
$ ./BasicIncome_Crackme
BasicIncome eligibility calculator
household_id> fea25b2266732462
Eligible. Voucher: SDG{9989f94654dedf39ee7330266ca05527}
```

Flag: SDG{9989f94654dedf39ee7330266ca05527}

Referensi = Claude

Catatan Belajar: Reverse Engineering

BasicIncome_Crackme — XOR Key Analysis

Tools: Ghidra, GDB, Python3 | Flag: SDG{9989f94654dedf39ee7330266ca05527}

1. Overview Challenge

BasicIncome_Crackme adalah binary ELF 64-bit yang berpura-pura menjadi kalkulator kelayakan program BasicIncome. Program meminta input berupa household_id sepanjang 16 karakter, memvalidasinya menggunakan operasi XOR, dan jika valid menampilkan voucher berupa flag.

Item	Detail
Nama Binary	BasicIncome_Crackme
Tipe File	ELF 64-bit (x86-64)
Tools Digunakan	Ghidra (decompiler), GDB (runtime debug), Python3 (kalkulasi XOR)
Teknik Utama	Static Analysis + Dynamic Analysis (Runtime Memory Dump)
Flag	SDG{9989f94654dedf39ee7330266ca05527}
Password	fea25b2266732462

2. Alur Analisis Step by Step

Step 1: Cek Strings Binary

Langkah pertama selalu cek strings binary untuk mencari petunjuk awal seperti pesan error, password hardcoded, atau nama fungsi:

```
strings ./BasicIncome_Crackme
```

Hasilnya: hampir tidak ada strings yang meaningful — tidak ada 'correct', 'wrong', atau password plaintext. Ini tanda binary kemungkinan menggunakan enkripsi/obfuscasi pada data pentingnya.

Step 2: Analisis Entry Point di Ghidra

Buka binary di Ghidra, pergi ke fungsi entry. Ditemukan pola standar:

```
FUN_00402b20(FUN_00401680, param_2, ...)
```

FUN_00402b20 adalah `__libc_start_main` — wrapper standar C runtime. FUN_00401680 adalah main function yang sesungguhnya. Entry point bukan tempat logika utama, selalu lompat ke fungsi yang dipassing sebagai argumen pertama ke `__libc_start_main`.

Step 3: Analisis Main Function (FUN_00401680)

Di dalam main, ditemukan logika validasi:

```
if ((&DAT_0047e0d0)[i] != ((&DAT_0047e0e0)[i % 8] ^ local_58[i]))
```

Dari sini bisa dibaca rumus validasinya:

```
DAT_0047e0d0[i] == DAT_0047e0e0[i % 8] XOR input[i]
```

Dibalik untuk mendapat input yang valid:

```
input[i] = DAT_0047e0d0[i] XOR DAT_0047e0e0[i % 8]
```

Selain itu ditemukan syarat penting: input harus tepat 16 karakter, jika tidak program langsung print 'Ineligible: household_id must be 16 characters.'

Step 4: Dump Data Statis dari Ghidra

Pergi ke alamat DAT_0047e0d0 di Listing view, select 16 bytes, klik kanan → Copy Special → Byte String (No Spaces):

```
DAT_0047e0d0 (16 bytes): de18ae735fbd79188e4bf87258eb7d18
```

Pergi ke alamat DAT_0047e0e0, ambil 8 bytes:

```
DAT_0047e0e0 (8 bytes, static): de18ae735fbd7918
```

Masalah ditemukan: nilai DAT_0047e0e0 sama persis dengan 8 bytes pertama DAT_0047e0d0. XOR dengan dirinya sendiri = selalu nol. Ini berarti data di 0047e0e0 belum di-populate saat binary dibaca statis — nilainya baru ada saat runtime.

Step 5: Runtime Memory Dump dengan GDB

Karena static analysis tidak cukup, gunakan GDB untuk dump memory saat program berjalan:

```
gdb ./BasicIncome_Crackme
(gdb) break *0x00401680
(gdb) run
(gdb) x/16bx 0x0047e0e0
```

Hasil runtime dump DAT_0047e0e0:

```
b87dcf416adf4b2a0000000000000000
```

Berbeda dari nilai statis! Ini konfirmasi bahwa key XOR di-populate secara dinamis saat program jalan — kemungkinan dari kalkulasi berbasis environment, waktu, atau nilai lain.

Step 6: Kalkulasi Password dengan Python

Dengan data runtime yang benar, hitung password:

```
d0 = bytes.fromhex('de18ae735fbd79188e4bf87258eb7d18')
e0 = bytes.fromhex('b87dcf416adf4b2a')
password = ''.join(chr(d0[i] ^ e0[i % 8]) for i in range(16))
# Output: fea25b2266732462
```

Step 7: Jalankan Program dan Dapat Flag

```
./BasicIncome_Crackme
household_id> fea25b2266732462
Eligible. Voucher: SDG{9989f94654dedf39ee7330266ca05527}
```

3. Memahami Operasi XOR dalam Crackme

Sifat Dasar XOR

XOR (exclusive OR) adalah operasi bitwise dengan sifat yang sangat berguna untuk enkripsi sederhana:

- $A \text{ XOR } B = C$
- $C \text{ XOR } B = A$ (bisa dibalik!)
- $A \text{ XOR } A = 0$ (XOR dirinya sendiri = selalu nol)
- $A \text{ XOR } 0 = A$ (XOR dengan nol = tidak berubah)

Kenapa Static Analysis Gagal di Kasus Ini

Nilai DAT_0047e0e0 di binary file (disk) berbeda dengan nilai saat runtime (memory). Ini teknik umum untuk mempersulit reverse engineering — data penting di-compute atau di-decrypt saat startup, sehingga tidak terlihat jika hanya membaca binary secara statis.

Solusinya: selalu kombinasikan static analysis (Ghidra) dengan dynamic analysis (GDB) untuk mendapat gambaran lengkap.

Pattern Validasi XOR yang Umum di Crackme

Pola yang ditemukan di challenge ini sangat umum:

- Ciphertext tersimpan di binary (DAT_0047e0d0)
- Key XOR tersimpan terpisah, bisa statis atau dinamis (DAT_0047e0e0)
- Input user di-XOR dengan key, hasilnya dibandingkan dengan ciphertext
- Password yang benar = ciphertext XOR key

4. Masalah yang Dihadapi

Tahap	Masalah	Solusi
Static Analysis	DAT_0047e0e0 sama dengan 8 bytes pertama DAT_0047e0d0 — XOR hasilnya semua nol	Data di-populate saat runtime, bukan dari file. Perlu GDB untuk dump memory saat program berjalan
Identifikasi Main	Entry point terlihat seperti infinite loop tanpa logika	Entry point adalah bootstrap — main yang sesungguhnya adalah argumen pertama ke __libc_start_main
Ghidra Hex View	Ghidra hanya detect 1 byte di DAT karena tidak tahu panjang data	Select manual N bytes dengan Shift+Click lalu Copy Special → Byte String (No Spaces)
Fungsi Salah	Terjebak menganalisis FUN_00408be0 yang ternyata fungsi internal runtime	Fokus pada fungsi yang jelas memiliki string output seperti 'Ineligible' atau 'Eligible'

5. Tools dan Teknik yang Digunakan

Ghidra

- Decompiler untuk membaca logika binary tanpa source code
- Listing view untuk melihat data di alamat memory tertentu
- Copy Special → Byte String untuk dump hex data

- Navigasi: tekan G lalu ketik alamat untuk lompat ke lokasi tertentu
- Klik kanan di decompiler → Go to DAT_xxxx untuk navigasi cepat

GDB

- Dynamic analysis — debug program saat berjalan
- `break *0xALAMAT` untuk set breakpoint di alamat spesifik
- `x/Nbx ALAMAT` untuk dump N bytes dalam format hex dari alamat tertentu
- Wajib dipakai ketika data penting di-compute saat runtime

Python3

- Kalkulasi XOR cepat tanpa perlu tools tambahan
- `bytes.fromhex()` untuk convert hex string ke bytes
- Operator `^` untuk XOR antar byte
- Gunakan `python3 << 'EOF' ... EOF` untuk script multi-line di terminal

6. Pelajaran Utama

1. Strings yang kosong bukan berarti binary mudah — justru tanda data di-obfuskasi
2. Entry point bukan main — selalu cari fungsi yang dipassing ke `__libc_start_main`
3. Static analysis saja tidak cukup — kombinasikan dengan GDB untuk data runtime
4. XOR adalah teknik enkripsi paling umum di crackme — pelajari sifat-sifatnya
5. Ghidra tidak tahu panjang data secara otomatis — select manual jika perlu
6. Kalau hasil XOR semua nol, kemungkinan besar salah key — cek dengan GDB
7. Python adalah teman terbaik untuk kalkulasi kriptografi sederhana di CTF

